🧱 Exercício 1 — Dockerfile + Compose: “Hello Container”

Objetivo:
Aprender a criar uma imagem customizada a partir de um Dockerfile e orquestrar com o docker-compose.

Descrição:
Crie uma aplicação simples (ex: Node.js, Python ou PHP) que exiba “Hello from Docker!”.
Os campistas devem:

1. Criar um **Dockerfile** que:
    - Use uma imagem base leve (ex: node:alpine, python:3.11-alpine ou php:8.2-apache).
    - Configure o WORKDIR.
    - Copie o código da aplicação.
    - Instale dependências.
    - Defina o comando de execução (CMD).
2. Criar um **docker-compose.yml** que:
    - Faça o **build** da imagem a partir do Dockerfile.
    - Exponha a porta (ex: 3000:3000).
    - Monte um **volume local** para permitir edição em tempo real (opcional).

Critérios de sucesso:

- docker compose up sobe a aplicação corretamente.
- Acessar http://localhost:3000 retorna a mensagem.
- Estrutura segue boas práticas (.dockerignore, imagem leve, etc).

🗃️ Exercício 2 — Compose com API + Banco de Dados

Objetivo:
Aprender a conectar múltiplos containers (API + DB) usando docker-compose e variáveis de ambiente.

Descrição:
Monte uma stack com dois serviços:

1. **api**
    - Build via Dockerfile (igual ao exercício anterior).
    - Usa variáveis de ambiente (DB_HOST, DB_USER, DB_PASS).
    - Se conecta ao banco via hostname do serviço db.
2. **db**
    - Use imagem oficial (postgres, mysql ou mongo).
    - Configure um volume para persistência.
    - Variáveis de ambiente no docker-compose.yml (ex: POSTGRES_PASSWORD, POSTGRES_DB).

Critérios de sucesso:

- docker compose up cria rede interna e os serviços se comunicam.
- A API consegue se conectar e gravar dados no banco.
- O volume persiste os dados mesmo após docker compose down.

Extras (avançado):

- Usar um arquivo .env externo para variáveis sensíveis.
- Testar dependência (depends_on).

⚙️ Exercício 3 — Boas Práticas + Múltiplos Ambientes

Objetivo:
Aplicar boas práticas de build e configuração de ambientes (dev/prod).

Descrição:
Usando o projeto do exercício anterior, os campistas devem:

1. Criar dois **Dockerfiles** ou estágios no mesmo arquivo:
    - dev: com volume montado e live reload (ex: nodemon, flask --reload).
    - prod: build otimizado, sem volumes, com imagem final leve (ex: FROM node:alpine + npm ci --only=production).
2. Criar dois **arquivos Compose**:
    - docker-compose.dev.yml — monta volume local, expõe portas, usa Dockerfile de dev.
    - docker-compose.prod.yml — usa imagem final otimizada, sem montar volumes.
3. Testar ambos os ambientes:
    - docker compose -f docker-compose.dev.yml up
    - docker compose -f docker-compose.prod.yml up

Critérios de sucesso:

- Ambientes dev/prod funcionam de forma independente.
- O build final segue boas práticas (camadas otimizadas, .dockerignore, imagem leve).
- Estrutura organizada e reutilizável.

🚀 Desafio Extra — Aplicação Completa com Docker Compose

Objetivo

Desenvolver e orquestrar uma aplicação **completa** utilizando **Dockerfile** e **Docker Compose**, aplicando boas práticas e estrutura profissional de containers.

Descrição do Desafio

Você deve montar uma stack composta por **três serviços principais**:

1. **API (backend)** – construída a partir de um Dockerfile
2. **Banco de dados** – utilizando imagem oficial (PostgreSQL ou MySQL)
3. **Interface de administração** – utilizando Adminer ou pgAdmin

A aplicação deve permitir que o usuário acesse a API e visualize/interaja com os dados do banco via interface web.

📦 Requisitos Técnicos

1️⃣ API

- Deve ser construída via **Dockerfile customizado**.
- Deve se conectar ao banco de dados através do **hostname do serviço** (db).
- Deve expor uma rota /health que retorne status da aplicação (ex: {"status": "ok"}).
- Variáveis de ambiente como DB_HOST, DB_USER, DB_PASS, DB_NAME devem ser configuradas no Compose.
- Use uma imagem base leve (node:alpine, python:3.11-alpine, etc.).
- O código-fonte deve ser montado via volume em ambiente de desenvolvimento.

2️⃣ Banco de Dados

- Utilize imagem oficial (postgres ou mysql).
- Configure **volume nomeado** para persistência.
- Configure variáveis de ambiente (usuário, senha e nome do banco).
- Adicione um **healthcheck** no Compose para garantir que o banco está disponível antes da API iniciar.

Exemplo:

```
healthcheck:
  test: ["CMD", "pg_isready", "-U", "postgres"]
  interval: 10s
  timeout: 5s
  retries: 5
```

3️⃣ Interface de Administração

- Utilize o adminer (para MySQL) ou pgadmin (para PostgreSQL).
- Configure o serviço para se conectar ao banco via hostname db.
- Exponha a porta (ex: 8080:80).

```
project/
│
├── api/
│   ├── Dockerfile
│   ├── package.json
│   └── src/
│       └── index.js
│
├── docker-compose.yml
└── .env
```

O que será avaliado

Organização e estrutura do projeto
Uso correto de variáveis de ambiente e .env
Uso de volumes e networks nomeadas
Configuração de dependências (depends_on, healthcheck)
Imagem leve e Dockerfile otimizado
Funcionamento geral da stack (docker compose up -d)

✅ Como entregar ?

Os exercícios devem ser entregues em repositório do GitHub através de aprovação de pull-request para a branch main.
A dinâmica fica da seguinte forma:

Criação de um repositório para entrega dos exercícios exemplo:: **exercicios-orion**
Criação de um pasta para cada conteúdo para melhor organização: docker/ e dentro do docker cada exercício proposto. Ex: Repositório: exercicios-orion - Diretório > Docker > exercicio-1 - exercicio-2 - exercicio-3 - exercicio-extra
Com os exercícios devidamente organizados e solucionados, deve-se enviar o link do repositório do Github que contém os exercícios para o mentor responsável pelo conteúdo.